explicit

Модификатор explicit применим только к конструктору с преобразованием типа и к функции преобразования типа.

Конструктор с таким модификатором explicit запрещает создание объекта с неявным преобразованием типа

Без explicit в конструкторе разрешается неявное приведение

```
class Point {
private:
    double x = 5.0;
    double y = 6.0;
public:
    Point() = default;
    Point(double x): x(x) {
        std::cout<< "Constructor" <<std::endl;
    }
    Point(const Point& s): x(s.x), y(s.y) {
        std::cout<< "Copy Constructor" <<std::endl;
    }
    Point operator + (const Point& o) {
        Point temp = o;
        temp.x+=x;
        temp.y+=y;
        return temp;
    }
   friend std::ostream& operator<< (std::ostream& out, const Point& obj){
       return out << obj.x << ' ' << obj.y << '\n';
    }
};

int main () {
    int x = 8, y = 4;
    Point p(x);
    std::cout << p + y << '\n';
};
```

explicit в конструкторе

```
class Point {
private:
    double x = 5.0;
    double y = 6.0;
public:
    Point() = default;
    explicit Point(double x): x(x) {
        std::cout<< "Constructor" <<std::endl;
    }
    Point(const Point& s): x(s.x), y(s.y) {
        std::cout<< "Copy Constructor" <<std::endl;
    }
    Point operator + (const Point& o) {
        Point temp = o;
        temp.x+=x;
        temp.y+=y;
        return temp;
    }
   friend std::ostream& operator<< (std::ostream& out, const Point& obj){
       return out << obj.x << ' ' << obj.y << '\n';
    }
};

int main () {
    int x = 8, y = 4;
    Point p(x);
    std::cout << p + y << '\n';
};
```

Invalid operands to binary expression ('Point' and 'int')

explicit в конструкторе

```
class Name {
    char* n = new char {};
public:
    Name() { std::cout << "Ctor Empty\n";}
    explicit Name(const char* n): n(new char[strlen(n) + 1]) {
        memcpy (this->n, n, strlen(n));
    }
    Name(const Name& other): n(new char[strlen(other.n) + 1]) {
        memcpy (this->n, other.n, strlen(other.n));
    }
    Name& operator=(const Name& other) = delete;
    ~Name() {
       if (n != nullptr) {
           delete [] n;
       }
    }
    friend Name operator+ (const Name& left, const Name& right);
};

Name operator+ (const Name& left, const Name& right) {
    Name left_copy = left;
    return strcat(left_copy.n, right.n);      ⊗ No viable conversion from returned value of type 'char *' to function return type 'Name'
}


int main () {
    Name name1;
    Name name2 = {"Bye"};                     2 ⊗ Chosen constructor is explicit in copy-initialization
    Name name3 = "Hello";                     ⊗ No viable conversion from 'const char [6]' to 'Name'
};
```

explicit в операторе преобразования

```
class Point {
private:
    double x = 5.0;
    double y = 6.0;
public:
    Point() = default;
    Point(int x): x(x) {
        std::cout<< "Constructor" <<std::endl;
    }
    Point(const Point& s): x(s.x), y(s.y) {
        std::cout<< "Copy Constructor" <<std::endl;
    }
    explicit operator double (){
        double z = x + 1.0*y/100;
        return z;
    }
};


int main () {
    double x = 8, y = 4;
    Point p(x);
    std::cout << static_cast<double>(p) + y << '\n';
};
```

Немножечко про ссылки и указатели

Норм, но надо следить за памятью

```
char* f () {
    char *t = new char[]{"Hello"};
    return t;
}
int main() {
    char* str = new char[]{"Hi"};
    std::cout << str;
    delete [] str;
    str = f();
    delete [] str;
}
```

Норм, но надо следить за памятью

```
char* f (char* t) {
    delete [] t;
    t = new char[]{"Hello"};
    return t;
}
int main() {
    char* str = new char[]{"Hi"};
    str = f(str);
    std::cout << str;
    delete [] str;
}
```

Выведется 1 символ, утечка динамической памяти:

```
char& f () {
    char* ret = new char[]{"Hello"};
    return *ret;
}
int main() {
    std::cout << f();
}
```

Подвисшая ссылка:

```
int& f () {
    int t = 6;
    return t;
}
int main() {
    int r = f();
    std::cout << r;
}
```

```
int& f (int t) {
    int temp(t);
    return t;
}
int main() {
    int r = 9;
    f(r);
}
```

Ок:

```
int& f (int &t) {
    t = 10;
    return t;
}
int main() {
    int r = f(r);
    std::cout << r;
}
```

Тожен норм, вывод: 18 18

```
int f (int* t) {
    int t1 = *t;
    t1+=9;
    return t1;
}
int main (){
    int r = 9;
    std::cout << f(&r) <<' ' << f(&r);
}
```

И ещё немножечко про перегрузку операторов

```
Programmer a;
Programmer b;
Programmer c;
Programmer d;
d = a + b + c;
```

Уничтожение временного объекта будет происходить в самом конце выражения, где он используется.

```
Programmer operator+ (const Programmer& left, const Programmer& right) {
    Programmer temp(left);
    temp+=right;
    return temp;
}
```

Таким образом, используется механизм продления жизни временного объекта, когда промежуточный результат присваивается константной ссылке (либо копируется по значению)

Перегрузка операторов new и delete

Существует возможность перегрузить операторы new, delete для более эффективного распределения динамической памяти. Всего существует 4 формы операторов выделения и освобождения памяти:

1. new - работа с одиночными объектами
2. new[] - работа с массивами объектов
3. delete - работа с одиночными объектами
4. delete[] работа с массивами объектов

Перегрузка new и delete (продолжение)

При перегрузке операторов для работы с динамической памятью нужно соблюдать следующие правила:

- операторам не требуется передавать параметр типа класс;
- первым параметром для new должен передаваться размер объекта size_t;
- возвращаемые значения new должны иметь тип void*;
- возвращаемые значения delete, delete[] должны иметь тип void;
- первый аргумент delete, delete[] должен иметь тип void*;
- данные операции являются статическими членами класса

Работа перегрузки new и delete

```
class Point {
. . .
public:
    void* operator new(size_t n);
    void operator delete (void* ptr);
. . .
};
```

При вызове оператора new сначала выделяется память для объекта:

- Если выделение прошло **успешно**, то **вызывается конструктор**.
- Если конструктор выбрасывает **исключение**, то выделенная **память освобождается**.

При вызове оператора delete все происходит в обратном порядке: сначала вызывается деструктор, потом освобождается память. **Деструктор не должен выбрасывать исключений**

Работа перегрузки new[] и delete[]

Когда оператор new[] используется для создания массива объектов, то сначала выделяется память для всего массива:

•Если выделение прошло **успешно**, то **вызывается конструктор** по умолчанию (или другой конструктор, если есть инициализатор) для каждого элемента массива начиная с нулевого.

•Если какой-нибудь конструктор выбрасывает **исключение**, то для всех созданных элементов массива **вызывается деструктор** в порядке, обратном вызову конструктора, затем выделенная память освобождается.

Для **удаления** массива надо вызвать оператор delete[], при этом для всех элементов массива вызывается деструктор в порядке, обратном вызову конструктора, затем выделенная память освобождается.

Пример

```
class Point {
        int x;
 . . .
};
void* Point::operator new(size_t n) {
    std::cout << n << std::endl; // 4
    return malloc(n);
}
void Point::operator delete (void* ptr) {
    free(ptr);
}
void* Point::operator new[](size_t n) {
    std::cout << n << std::endl;  //Если есть деструктор, будет 28 (24)
                                  //Если деструктора нет, 20 — но неточно ☺
    return malloc(n);
}
void Point::operator delete[] (void* ptr) {
    free(ptr);
}
int main (){
    Point* test = new Point [5];
    Point* test1 = new Point(3,3);
    delete [] test; delete test1;
}
```

Инициализатор для массива

```
#include <iostream>


class Point {
    int x, y;
public:
    Point() : x(0), y(0) { std::cout << "Default" << std::endl; }
    Point(int val) : x(val), y(val) { std::cout << "Single" << std::endl; }
    Point(int a, int b) : x(a), y(b) { std::cout << "Two args" << std::endl; }
};


int main() {
    Point* p1 = new Point[3];                    // Default, Default, Default
    Point* p2 = new Point[3]{1};                  // Single, Default, Default
    Point* p3 = new Point[3]{1, {2,3}};           // Single, Two args, Default
    Point* p4 = new Point[3]{{1,2}, 3, Point(4)}; // Two args, Single, Single
}
```

Указатели на члены класса

```
class Box {
public:
    int width = 5;
    int height = 8;
    Box() {}
    Box(int w, int h): width (w), height (h) {}
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};

int main () {
    int Box::*p;
    p = &Box::width;
    Box b(6,7);
    b.*p = 4;
    void (Box::*pf)(int);
    pf = &Box::SetWidth;
    (b.*pf)(3);
    pf = &Box::SetHeight;
    (b.*pf)(5);
    Box b1(1,2);
    (b1.*pf)(10);
};
```

Наследование

Отношения между классами

- **Ассоциация**. Ассоциация – это когда один класс включает в себя другой класс в качестве одного из полей. Ассоциация описывается словом «имеет». Например, автомобиль имеет двигатель.
- **Наследование**. Наследование подразумевает, что один объект является разновидностью другого, наследуется, то есть «перенимает» его свойства, добавляя свои. Наследование описывается словом «является». Легковой автомобиль является автомобилем.

Ассоциация. Композиция и агрегация.

Композиция – это когда двигатель не существует отдельно от автомобиля. Он создается при создании автомобиля и полностью управляется автомобилем.

В типичном примере **экземпляр** двигателя **будет создаваться в конструкторе** автомобиля.

```
class Engine {
    int power;
public:
    Engine(int p): power(p) { }
};


class Car {
    const char* model = "Porshe";
    Engine* engine;
public:
    Car(): engine(new Engine(360)) {}
};
```

Ассоциация. Композиция и агрегация.

Агрегация – это когда экземпляр двигателя создается где-то в другом месте кода, и передается в конструктор автомобиля в качестве параметра.

```
class Engine {
    int power;
public:
    Engine(int p): power(p) { }
};

class Car {
    const char* model = "Porshe";
    Engine* engine;
public:
    Car(Engine *SomeEngine): engine(SomeEngine) {}
};

int main () {
    Engine* goodEngine = new Engine(360);
    Car* porshe = new Car(goodEngine);
}
```

Наследование

Можно сказать, что наследование - это всего лишь **особая форма композиции**. Производные классы состоят из своих базовых классов.

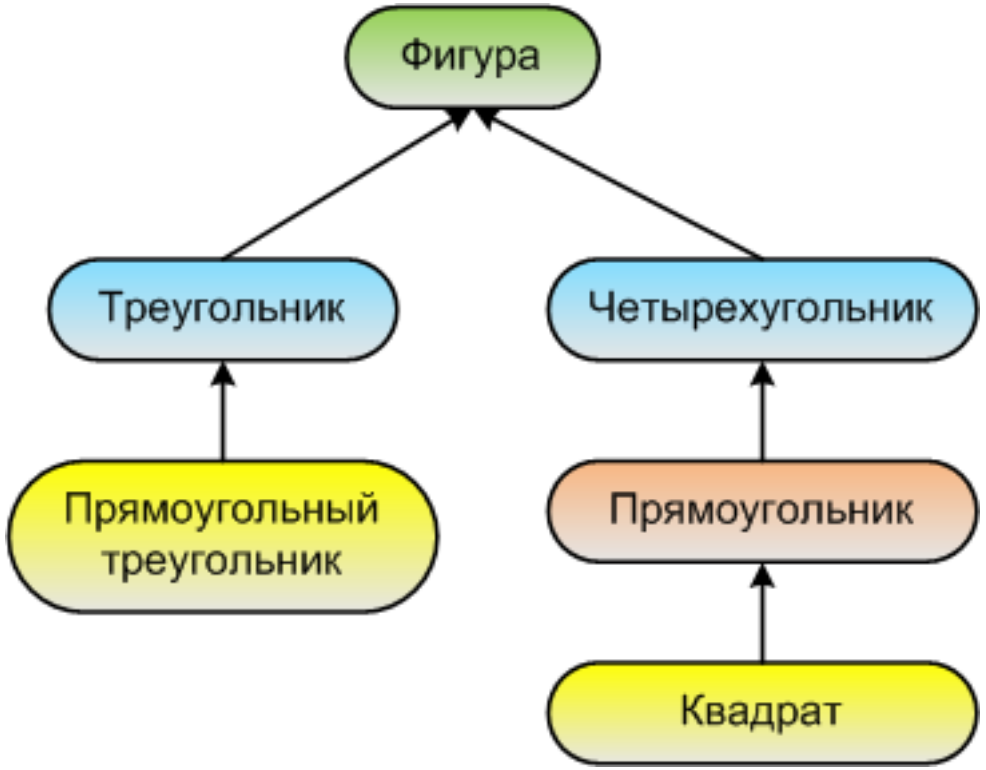
Однако, связь производного с базовым гораздо более тесная, чем связь композита с компонентом.

Наследование - механизм передачи свойств одних классов другим классам в процессе проектирования иерархии классов. Исходные классы называют:

- **базовыми** (БК) (родителями);
- **производными** (ПК) (потомками).

Дерево наследования

Для удобного изображения отношений классов при наследовании строят специальный граф: **дерево наследования**:



Базовые классы изображают над производными.

Пример

```
156 struct Box {  // тип "коробка"
157     int _width = 0;
158     int _height = 0;
159     void SetWidth(int w) { _width = w; }
160     void SetHeight(int h) { _height = h; }
161 };
162 struct ColoredBox : Box { // "цветная коробка"
163     int _color;
164     void SetColor(int c) {
165         _color = c;
166     }
167 };
168
169 int main () {
170     ColoredBox cbox;
171     cbox.SetColor(5);
172     std::cout << cbox._color << ' ' << cbox._height << cbox._width;
173 };
174
175
```

Lections 〉 Thread 1 〉 0 main

- cbox (ColoredBox)
  - Box
    - _width = (int) 0
    - _height = (int) 0
  - _color = (int) 5

```
5 00(lldb)
```

Модификаторы доступа

При описании производного класса в его заголовке перечисляются все базовые для него классы. Доступ к элементам этих классов регулируется с помощью модификаторов доступа:

- private;
- protected;
- public.

Если базовых классов несколько, они перечисляются через запятую.

Кроме этого, учитываются модификаторы доступа в самом классе ☺

Модификатор доступа

Модификатор доступа **не влияет на прямое взаимоотношение между производным и базовым классом**. Прямому наследнику с любым ключом доступа будут доступны все public и protected поля. При любом модификаторе доступа private-поля будут закрыты. То же касается и методов.

```
class Box {  // тип "коробка"
    int width = 0;
    int height = 0;
protected:
    int length = 0;
public:
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};

class ColoredBox : private Box { // "цветная коробка"
    int color = 8;
public:
    void SetColor(int c, int len, int w, int h) {
        width = 6;                          'width' is a private member of 'Box'
        height = 7;                         'height' is a private member of 'Box'
        length = len;  //Разрешено
        SetWidth(w);    //Разрешено
        SetHeight(h);   //Разрешено
        color = c;
    }
};
```

```
class Box {  // тип "коробка"
    int width = 0;
    int height = 0;
protected:
    int length = 0;
public:
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};

class ColoredBox : public Box { // "цветная коробка"
    int color = 8;
public:
    void SetColor(int c, int len, int w, int h) {
        width = 6;                                  'width' is a priva
        height = 7;                                 'height' is a priva
        length = len;  //Разрешено
        SetWidth(w);    //Разрешено
        SetHeight(h);   //Разрешено
        color = c;
    }
};
```

```
class Box {  // тип "коробка"
    int width = 0;
    int height = 0;
protected:
    int length = 0;
public:
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};

class ColoredBox : protected Box { // "цветная коробка"
    int color = 8;
public:
    void SetColor(int c, int len, int w, int h) {
        width = 6;                                  'width' is a private
        height = 7;                                 'height' is a private
        length = len;  //Разрешено
        SetWidth(w);    //Разрешено
        SetHeight(h);   //Разрешено
        color = c;
    }
};
```

Модификатор доступа влияет на доступность соответствующих полей из «сторонней» функции или следующего наследника

Модификатор доступа public

Из main у производного класса видны открытые поля базового класса.

Доступ к protected запрещён (к private, очевидно, тоже).

```
class Box {  // тип "коробка"
    int width = 0;
    int height = 0;
protected:
    int length = 0;
public:
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};

class ColoredBox : public Box { // "цветная коробка"
    int color = 8;
public:
    void SetColor(int c, int len, int w, int h) {
        length = len;  //Разрешено
        SetWidth(w);    //Разрешено
        SetHeight(h);   //Разрешено
        color = c;
    }
};

int main () {
    ColoredBox cbox;
    cbox.SetWidth(5); //Разрешено
    cbox.length = 6; //Запрещено
};
```

'length' is a protected member of 'Box'

Модификатор доступа public (продолжение)

```
class Box {  // тип "коробка"
    int width = 0;
    int height = 0;
protected:
    int length = 0;
public:
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};
class ColoredBox : public Box { // "цветная коробка"
    int color = 8;
public:
    void SetColor(int c, int len, int w, int h) {
        length = len;  //Разрешено
        SetWidth(w);    //Разрешено
        SetHeight(h);   //Разрешено
        color = c;
    }
};
class ExColoredBox: public ColoredBox {
    int print = 3;
public:
    void setPrint(int p, int len) {
        print = p; length = len;
    }
};

int main () {
    ExColoredBox exbox;
    exbox.SetWidth(5);
    exbox.length = 8;
};
```

'length' is a protected member of 'Box'

Модификатор доступа protected

Все поля базового класса становятся недоступны при попытке вызова из main (к private даже не пытаемся обратиться).

```
class Box {  // тип "коробка"
    int width = 0;
    int height = 0;
protected:
    int length = 0;
public:
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};
class ColoredBox : protected Box { // "цветная коробка"
    int color = 8;
public:
    void SetColor(int c, int len, int w, int h) {
        length = len;  //Разрешено
        SetWidth(w);    //Разрешено
        SetHeight(h);   //Разрешено
        color = c;
    }
};

int main () {
    ColoredBox cbox;
    cbox.SetWidth(5);          'SetWidth' is a protected member of 'Box'
    cbox.length = 8;           'length' is a protected member of 'Box'
};
```

```
class Box {  // тип "коробка"
    int width = 0;
    int height = 0;
protected:
    int length = 0;
public:
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};
class ColoredBox : protected Box { // "цветная коробка"
    int color = 8;
public:
    void SetColor(int c, int len, int w, int h) {
        length = len;  //Разрешено
        SetWidth(w);    //Разрешено
        SetHeight(h);   //Разрешено
        color = c;
    }
};
class ExColoredBox: public ColoredBox {
    int print = 3;
public:
    void setPrint(int p, int len) {
        print = p;
        length = len;
    }
};
int main () {
    ExColoredBox exbox;
    exbox.SetColor(5,6,7,8);
    exbox.SetWidth(7);
};
```

'SetWidth' is a protected member of 'Box'

Модификатор доступа protected (продолжение)

Модификатор доступа private

При таком модификаторе доступа у производного класса при вызове из main вообще недоступны поля базового.

```
class Box {  // тип "коробка"
    int width = 0;
    int height = 0;
protected:
    int length = 0;
public:
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};
class ColoredBox : private Box { // "цветная коробка"
    int color = 8;
public:
    void SetColor(int c, int len, int w, int h) {
        length = len;
        SetWidth(w);
        SetHeight(h);
        color = c;
    }
};

int main () {
    ColoredBox cbox;
    cbox.SetWidth(5);
    cbox.length = 8;
};
```

'SetWidth' is a private member of 'Box'

'length' is a private member of 'Box'

Модификатор доступа private (продолжение)

```
class Box {  // тип "коробка"
    int width = 0;
    int height = 0;
protected:
    int length = 0;
public:
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};
class ColoredBox : private Box { // "цветная коробка"
    int color = 8;
public:
    void SetColor(int c, int len, int w, int h) {
        length = len;
        SetWidth(w);
        SetHeight(h);
        color = c;
    }
};
class ExColoredBox: public ColoredBox {
    int print = 3;
public:
    void setPrint(int p, int len) {
        print = p;
        length = len;                    'length' is a private member of 'Box'
    }
};
int main () {
    ExColoredBox exbox;
    exbox.SetColor(5,6,7,8);
    exbox.SetWidth(7);                   'SetWidth' is a private member of 'Box'
};
```

Пример

```
185 class A {
186     int y = 0;
187 protected:
188     int x = 1;
189 public:
190     int z = 2;
191 };
192 class B: protected A {
193     int xb = 3;
194 public:
195     int yb = 4;
196 };
197 class C: private B {
198 public:
199     void f() {
200         x = 6;
201         std::cout << x << ' ' << z << ' ' << yb << '\n';
202     }
203     int xc = 6;
204 };
205 int main () {
206     B b;
207     std::cout << b.yb;
208     C c;
209     c.f();
210 };
```

Lections 〉 Thread 1 〉 0 main

```
b (B)
  A
    y = (int) 0
    x = (int) 1
    z = (int) 2
  xb = (int) 3
  yb = (int) 4
c (C)
  B
    A
      y = (int) 0
      x = (int) 1
      z = (int) 2
    xb = (int) 3
    yb = (int) 4
  xc = (int) 6
```

```
4(lldb)
```

Задача

```
class A {
    int y = 0;
protected:
    int x = 1;
public:
    int z = 2;
};


class B: A {
    int xb = 3;
public:
    int yb = 4;
    void f(A& a) {
        a.x = 2; //Недоступно, к классу обращаемся как к «внешнему»
        x = 2;  //Разрешено
    }
};


int main () {
    B b;
    std::cout << b.yb;
};
```